

LAXMI CHARITABLE TRUST'S
SHETH L.U.J. COLLEGE OF ARTS &
SIR M.V. COLLEGE OF SCIENCE AND COMMERCE
Department of Information Technology (B.Sc. IT Semester V)

APPLIED BUSINESS ANALYTICS

BUSINESS ANALYTICS PROJECT

Business Analytics for Retail and Warehouse Sales

Project Details	Information
Student Name	Lavkush Vishwakarma
Roll No.	T048
Class / Batch	TYIT
Subject	Applied Business Analytics
Academic Year	2026–2027
Project Title	Business Analytics for Retail and Warehouse Sales

2. Table of Contents

- 3. Introduction
- 4. Problem Statement
- 5. Objectives
- 6. Business Scenario / Domain Description
- 7. Dataset Information
- 8. Data Collection
- 9. Data Preprocessing
- 10. Exploratory Data Analysis
- 11. Data Visualization
- 12. Descriptive Statistical Analysis
- 13. Distribution Analysis
- 14. Correlation Analysis
- 15. Regression Analysis
- 16. Probability Distributions
- 17. Decision-Making Under Uncertainty
- 18. Sampling and Sampling Distribution
- 19. Confidence Interval Estimation
- 20. Bootstrapping
- 21. Hypothesis Testing
- 22. A/B Testing Design
- 23. Time-Series Analysis
- 24. Forecast Accuracy
- 25. Optimization Modeling
- 26. Simulation Modeling
- 27. ANOVA
- 28. Experimental Design
- 29. Factorial Experiment
- 30. Business Intelligence Dashboard
- 31. Business Insights and Findings
- 32. Business Recommendations
- 33. Conclusion
- 34. Appendix

3. Introduction

Business Analytics is the process of collecting, cleaning, analyzing, and interpreting data to support better business decisions. It combines data visualization, descriptive statistics, probability, statistical testing, predictive modelling, and optimization to turn raw records into useful business information.

This project focuses on retail and warehouse sales. Sales operations must understand which item categories contribute most to sales, how sales values vary, whether retail sales are associated with warehouse activity, and how historical patterns can support inventory and planning decisions.

The project uses Python libraries such as Pandas, NumPy, Matplotlib, Seaborn, SciPy, scikit-learn, and Statsmodels.

Results must be taken from the supplied CSV and the accompanying Jupyter Notebook. Screenshot spaces have been intentionally left throughout this report.

4. Problem Statement

Retail and warehouse datasets may contain many records, categories, and numerical measures. Without analysis, it is difficult to identify high-performing item types, understand sales variability, evaluate relationships between retail and warehouse activity, or make informed inventory decisions.

Problem Statement: To analyze retail and warehouse sales data using Business Analytics techniques in order to identify category-wise performance, data patterns, relationships among variables, and evidence-based opportunities for improving reporting and inventory planning.

5. Objectives

1. To load, inspect, clean, and validate the retail and warehouse sales dataset.
2. To perform exploratory data analysis and summarize retail sales by item type.
3. To create suitable charts for comparing sales totals, averages, and distributions.
4. To calculate descriptive statistics and examine relationships among numerical variables.
5. To apply regression, probability, sampling, confidence intervals, bootstrapping, and hypothesis testing where appropriate.
6. To aggregate monthly sales and evaluate forecasting methods only when enough time periods exist.
7. To demonstrate inventory optimization and simulation using clearly stated assumptions.
8. To perform one-way ANOVA and assess whether factorial ANOVA is supported by the available data.
9. To present business insights and recommendations supported by actual output.

6. Business Scenario / Domain Description

Business Name

Retail and Warehouse Sales Operations (academic case study; no specific real company is assumed).

Type of Business

Retail distribution and warehouse operations.

Products / Services Offered

Multiple item types recorded in the dataset. Confirm the exact category names from the CSV.

Target Customers

Retail buyers and business customers served by the sales operation.

Business Operations

The operation records retail sales, warehouse sales and/or retail transfers. Analytics can support category review, inventory planning, sales monitoring, and reporting.

Important Business Problems

- Which item types contribute the greatest total retail sales?
- How do average sales and variability differ across item types?
- Are retail sales associated with warehouse sales or retail transfers?
- Does the available data contain enough time periods for meaningful forecasting?
- Do the item types show statistically significant differences in mean retail sales?
- How can inventory levels be compared under transparent cost assumptions?

7. Dataset Information

Dataset name: Retail and Warehouse Sales CSV. Use the exact filename supplied with the project. The actual record count, number of columns, and time coverage must be copied from the notebook output.

Variable	Description	Variable Type
YEAR	Year of the record, if present	Numerical / time
MONTH	Month of the record, if present	Numerical / time
ITEM TYPE	Product/item category	Categorical
RETAIL SALES	Retail sales measure	Numerical

RETAIL TRANSFERS	Retail transfer measure, if present	Numerical
WAREHOUSE SALES	Warehouse sales measure, if present	Numerical
SUPPLIER / ITEM CODE / DESCRIPTION	Identifiers or descriptions, if present	Categorical / identifier

Confirm all variable names and data types against the actual CSV; optional columns may differ.

Dataset: Retail and warehouse Sale - Retail and warehouse Sale.csv
Shape: (30000, 9)

	YEAR	MONTH	SUPPLIER	ITEM CODE	ITEM DESCRIPTION	ITEM TYPE	RETAIL SALES	RETAIL TRANSFERS	WAREHOUSE SALES
0	2020	1	REPUBLIC NATIONAL DISTRIBUTING CO	100009	BOOTLEG RED - 750ML	WINE	0.00	0.0	2.0
1	2020	1	PWSWN INC	100024	MOMENT DE PLAISIR - 750ML	WINE	0.00	1.0	4.0
2	2020	1	RELIABLE CHURCHILL LLLP	1001	S SMITH ORGANIC PEAR CIDER - 18.7OZ	BEER	0.00	0.0	1.0
3	2020	1	LANTERNA DISTRIBUTORS INC	100145	SCHLINK HAUS KABINETT - 750ML	WINE	0.00	0.0	1.0
4	2020	1	DIONYSOS IMPORTS INC	100293	SANTORINI GAVALA WHITE - 750ML	WINE	0.82	0.0	0.0

Rows: 30000
Columns: 9

Column names:
['YEAR', 'MONTH', 'SUPPLIER', 'ITEM CODE', 'ITEM DESCRIPTION', 'ITEM TYPE', 'RETAIL SALES', 'RETAIL TRANSFERS', 'WAREHOUSE SALES']

Data types:

	Data Type
YEAR	int64
MONTH	int64
SUPPLIER	object
ITEM CODE	object
ITEM DESCRIPTION	object
ITEM TYPE	object
RETAIL SALES	float64
RETAIL TRANSFERS	float64
WAREHOUSE SALES	float64

Missing values:

	Missing Values
YEAR	0
MONTH	0
SUPPLIER	33
ITEM CODE	0
ITEM DESCRIPTION	0
ITEM TYPE	0
RETAIL SALES	1
RETAIL TRANSFERS	0
WAREHOUSE SALES	0

Duplicate records: 0

8. Data Collection

The dataset is imported from a CSV file using Pandas. In the final submission, provide the original dataset source or download URL, collection method, time period, and any required attribution. Do not describe the dataset as collected from a live business unless this is verified.

Data Collection Method

The CSV file is loaded into Python using the Pandas `read_csv()` function.

Number of Records and Time Period

Record count and coverage period are to be filled from the notebook output.

```
Original rows: 30000
Rows available for retail-sales analysis: 29999
Rows removed for missing ITEM TYPE or RETAIL SALES: 1
```

9. Data Preprocessing

Data preprocessing includes checking missing values and duplicate records, converting numerical fields to numeric types, removing unnecessary whitespace from labels, and excluding rows missing the fields required for a particular analysis. Any rows removed should be reported. Extreme values should be investigated rather than automatically deleted.

Original rows: 30000
 Rows available for retail-sales analysis: 29999
 Rows removed for missing ITEM TYPE or RETAIL SALES: 1

	YEAR	MONTH	SUPPLIER	ITEM CODE	ITEM DESCRIPTION	ITEM TYPE	RETAIL SALES	RETAIL TRANSFERS	WAREHOUSE SALES
0	2020	1	REPUBLIC NATIONAL DISTRIBUTING CO	100009	BOOTLEG RED - 750ML	WINE	0.00	0.0	2.0
1	2020	1	PWSWN INC	100024	MOMENT DE PLAISIR - 750ML	WINE	0.00	1.0	4.0
2	2020	1	RELIABLE CHURCHILL LLLP	1001	S SMITH ORGANIC PEAR CIDER - 18.7OZ	BEER	0.00	0.0	1.0
3	2020	1	LANTERNA DISTRIBUTORS INC	100145	SCHLINK HAUS KABINETT - 750ML	WINE	0.00	0.0	1.0
4	2020	1	DIONYSOS IMPORTS INC	100293	SANTORINI GAVALA WHITE - 750ML	WINE	0.82	0.0	0.0

10. Exploratory Data Analysis

Exploratory Data Analysis (EDA) is used to understand the dataset before statistical modelling. It includes viewing the first and last records, checking dimensions and data types, identifying missing values, and reviewing the available item categories.

Unique item types:
 ['BEER', 'DUNNAGE', 'KEGS', 'LIQUOR', 'NON-ALCOHOL', 'REF', 'STR_SUPPLIES', 'WINE']

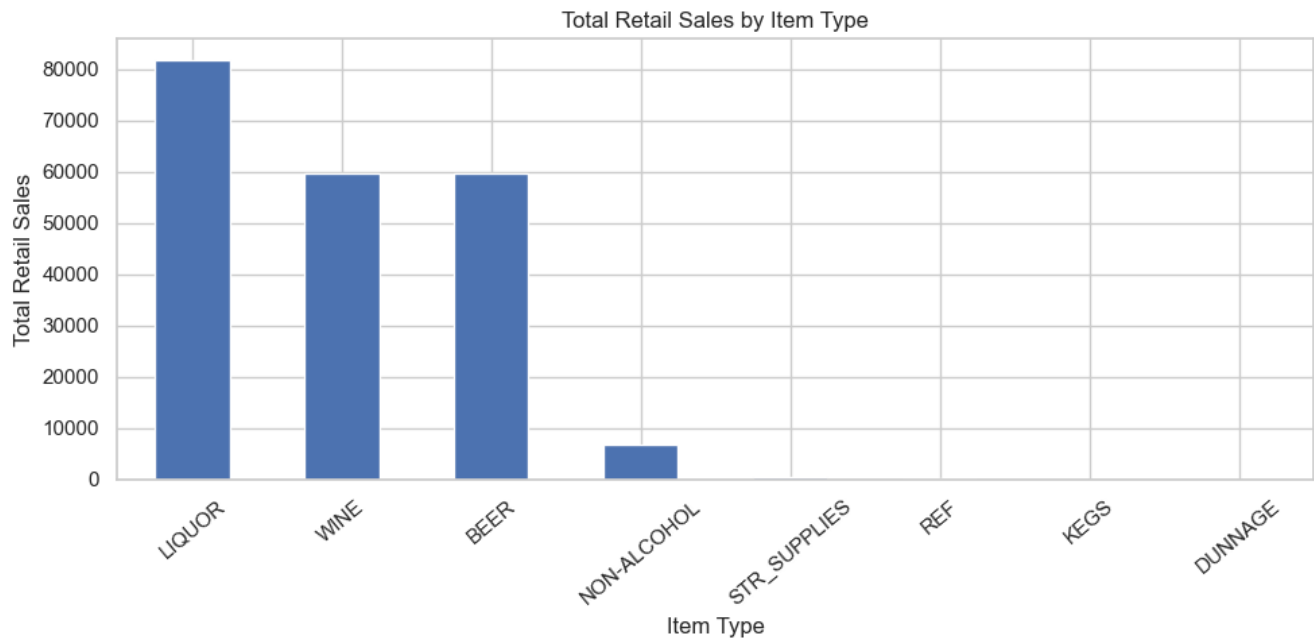
	count	mean	median	total	std
ITEM TYPE					
LIQUOR	5995	13.635171	2.52	81742.85	38.339506
WINE	18680	3.195334	0.00	59688.84	10.851343
BEER	4217	14.118748	0.00	59538.76	55.172342
NON-ALCOHOL	215	31.742419	5.26	6824.62	194.053302
STR_SUPPLIES	63	5.485714	0.73	345.60	9.724714
REF	8	5.783750	0.00	46.27	8.830771
KEGS	808	0.000000	0.00	0.00	0.000000
DUNNAGE	13	0.000000	0.00	0.00	0.000000

11. Data Visualization

Suitable charts are used to communicate category performance, data spread, and relationships between numerical variables. Replace each placeholder with the actual chart generated by the notebook.

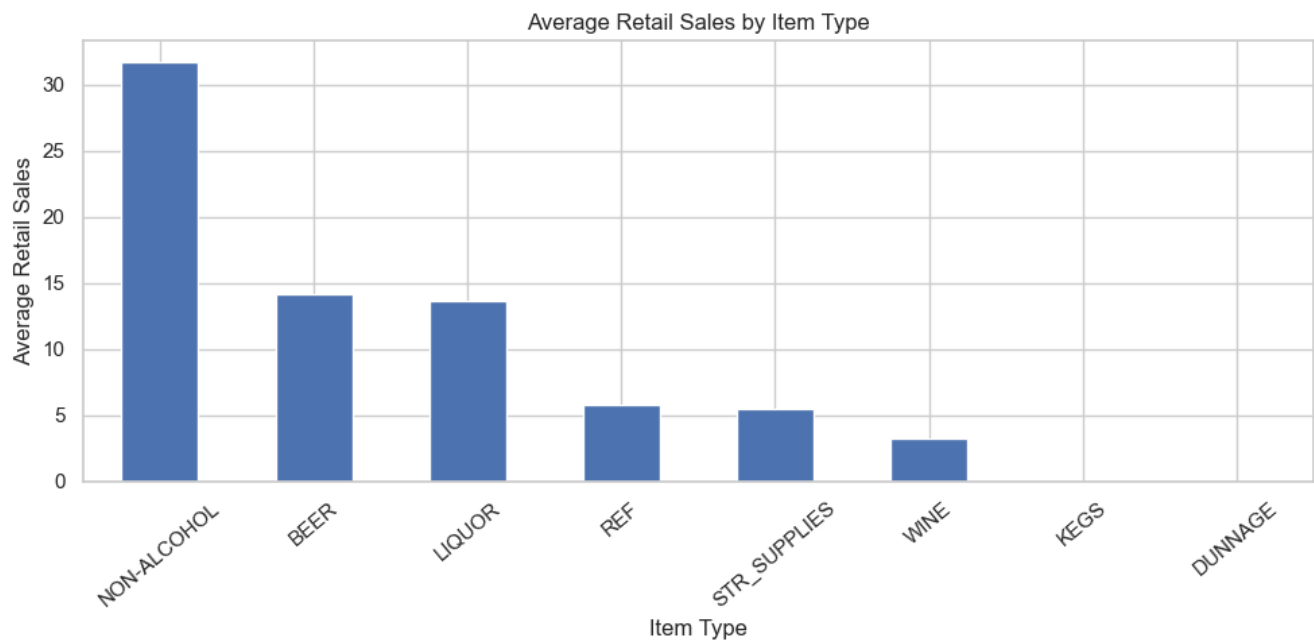
1. Bar Chart – Total Retail Sales by Item Type

Shows the total retail sales attributed to each item category.



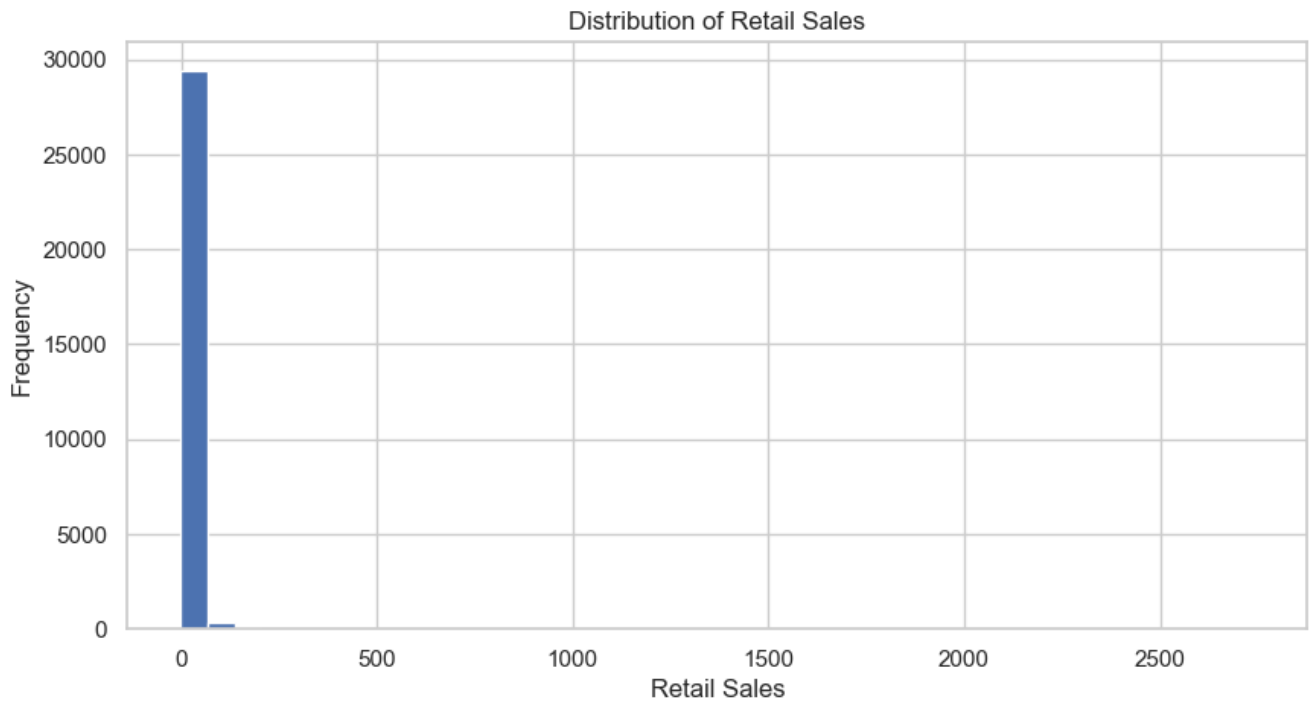
2. Bar Chart – Average Retail Sales by Item Type

Compares the average retail sales value for each category.



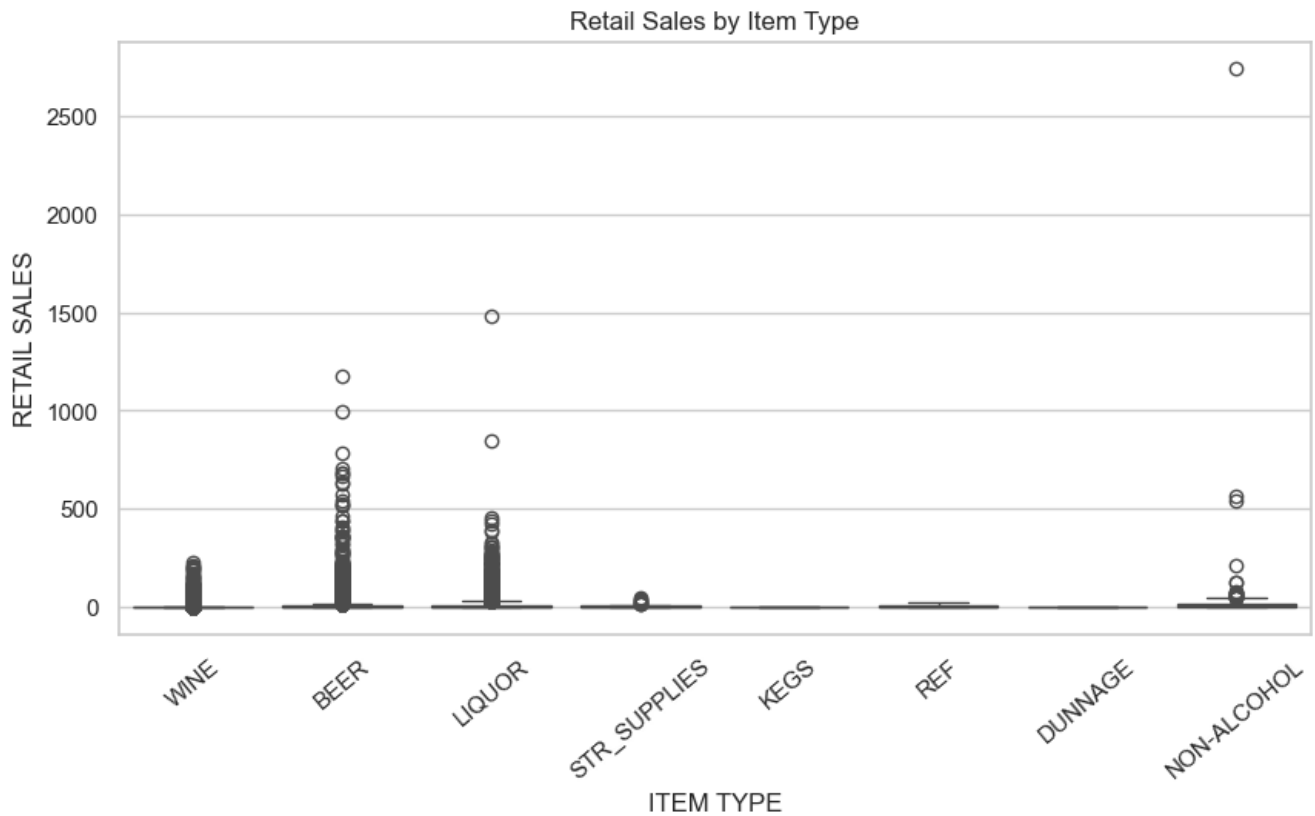
3. Histogram – Distribution of Retail Sales

Shows the frequency distribution and possible skewness of retail sales.



4. Box Plot – Retail Sales by Item Type

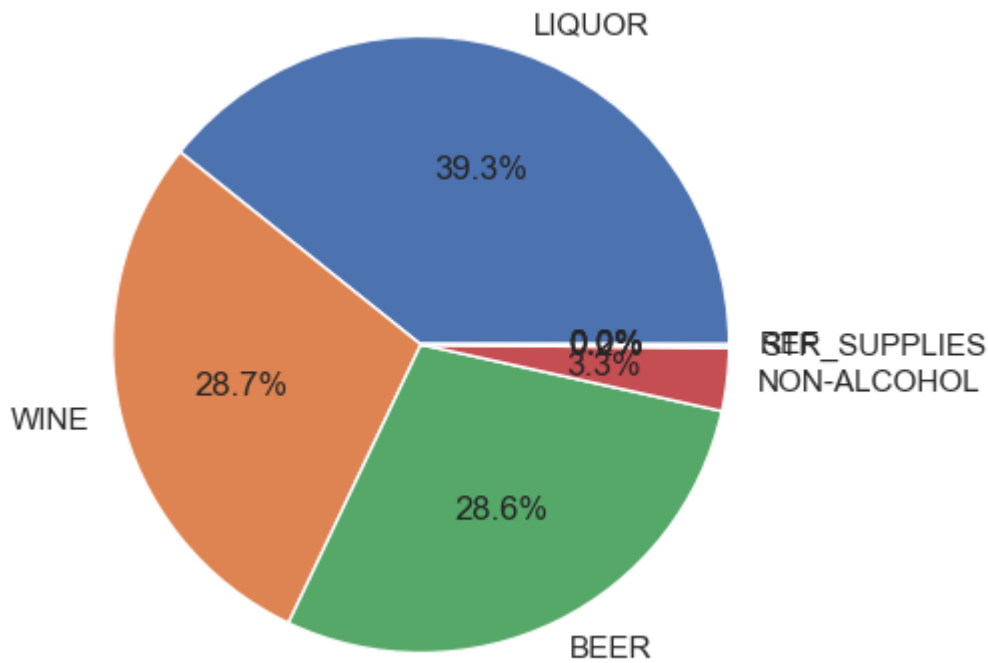
Compares spread, median, and potential outliers across categories.



5. Pie Chart – Sales Share by Item Type

Shows each item type's share of total retail sales.

Share of Total Retail Sales by Item Type



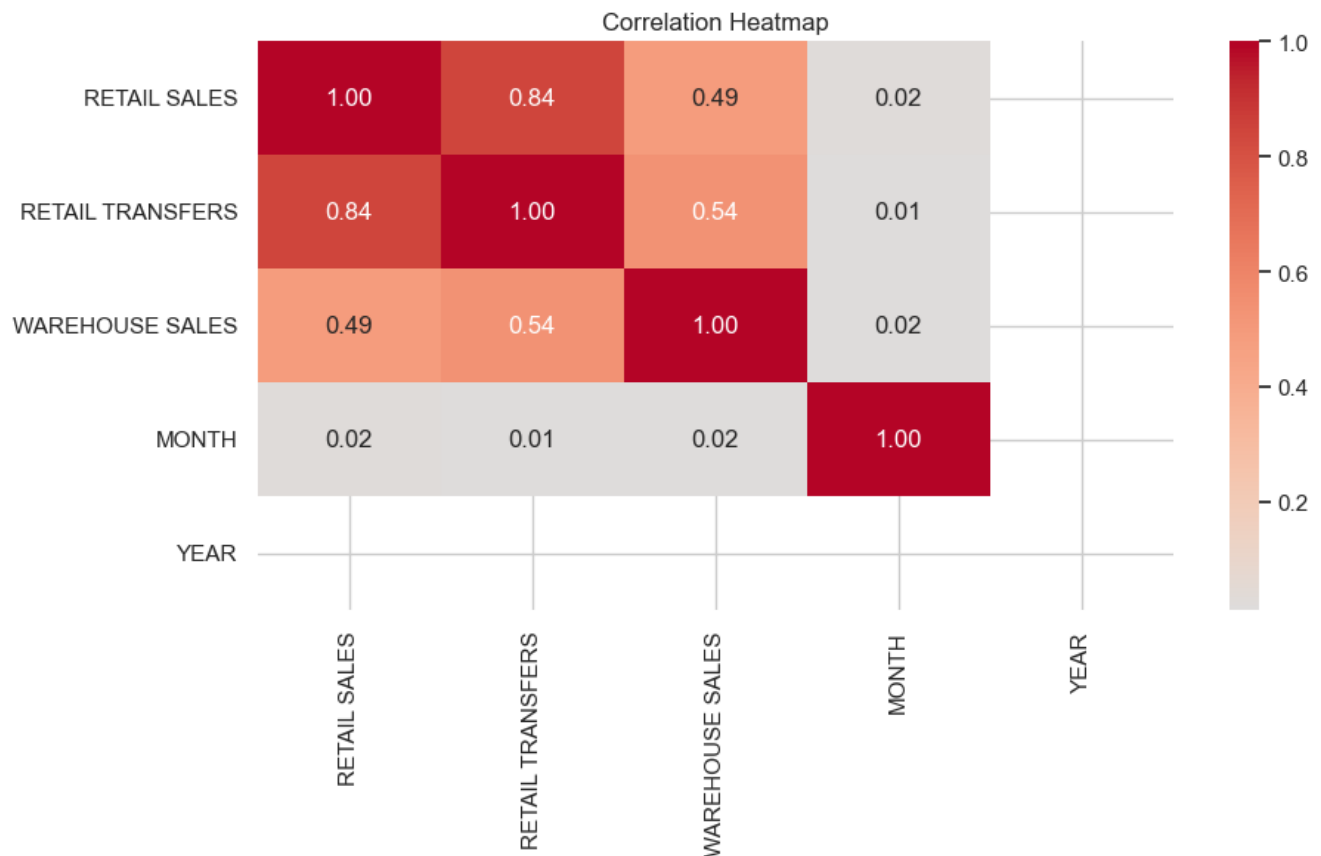
6. Scatter Plot – Retail Sales vs Warehouse Sales

Examines a possible relationship when both variables are available.



7. Heatmap – Correlation Matrix

Summarizes pairwise correlations among available numeric variables.



12. Descriptive Statistical Analysis

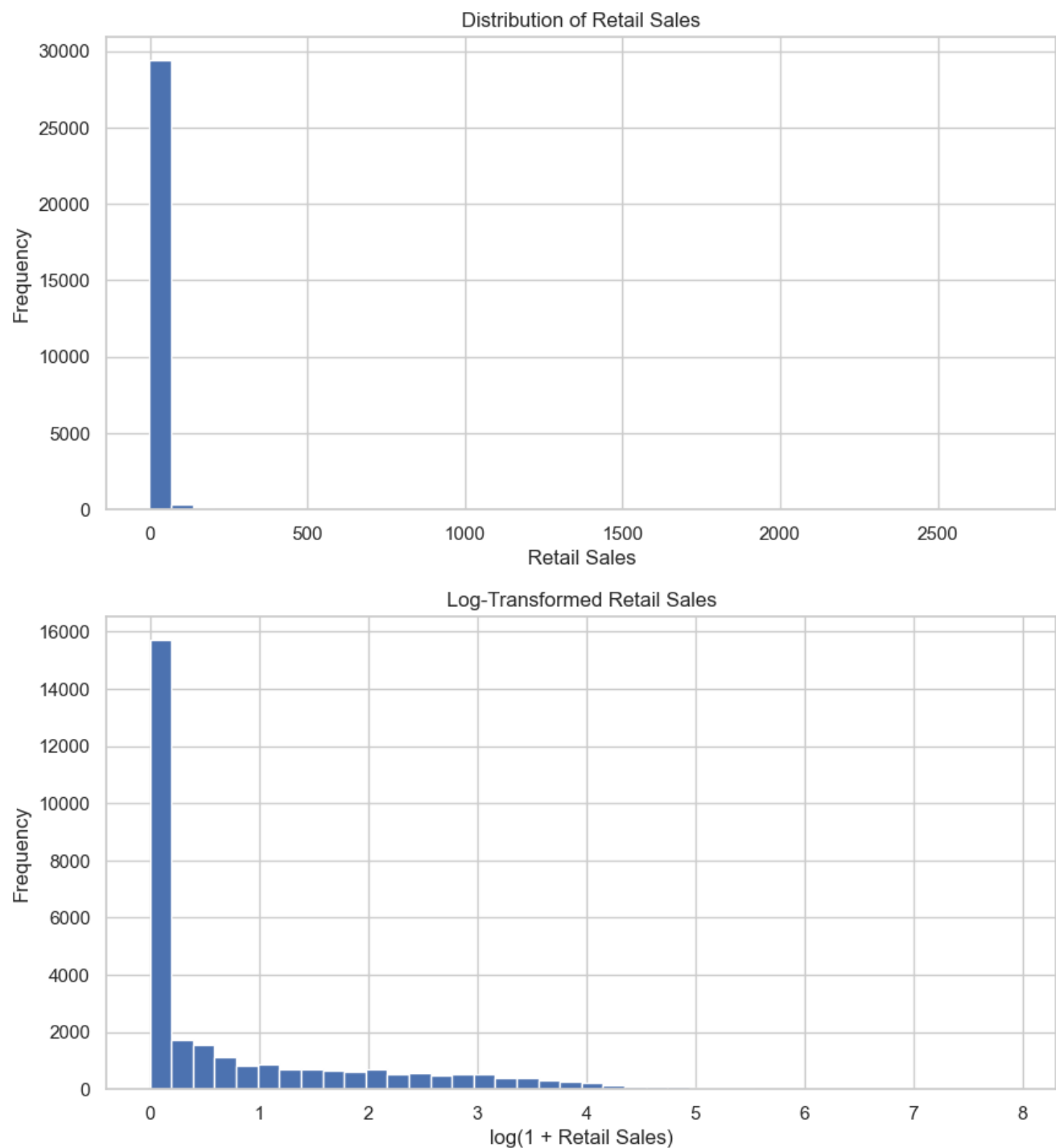
Descriptive statistics summarize the central tendency, spread, and shape of retail sales. The notebook calculates count, mean, median, mode, minimum, maximum, range, variance, standard deviation, and skewness. Category summaries include count, mean, median, total, and standard deviation.

	Value
Count	29999.000000
Mean	6.939796
Median	0.160000
Mode	0.000000
Minimum	-0.420000
Maximum	2739.000000
Range	2739.420000
Variance	1094.356120
Standard deviation	33.081054
Skewness	30.230696

	RETAIL SALES	RETAIL TRANSFERS	WAREHOUSE SALES	MONTH	YEAR
RETAIL SALES	1.000000	0.844597	0.491822	0.024374	NaN
RETAIL TRANSFERS	0.844597	1.000000	0.540071	0.011052	NaN
WAREHOUSE SALES	0.491822	0.540071	1.000000	0.017212	NaN
MONTH	0.024374	0.011052	0.017212	1.000000	NaN
YEAR	NaN	NaN	NaN	NaN	NaN

13. Distribution Analysis

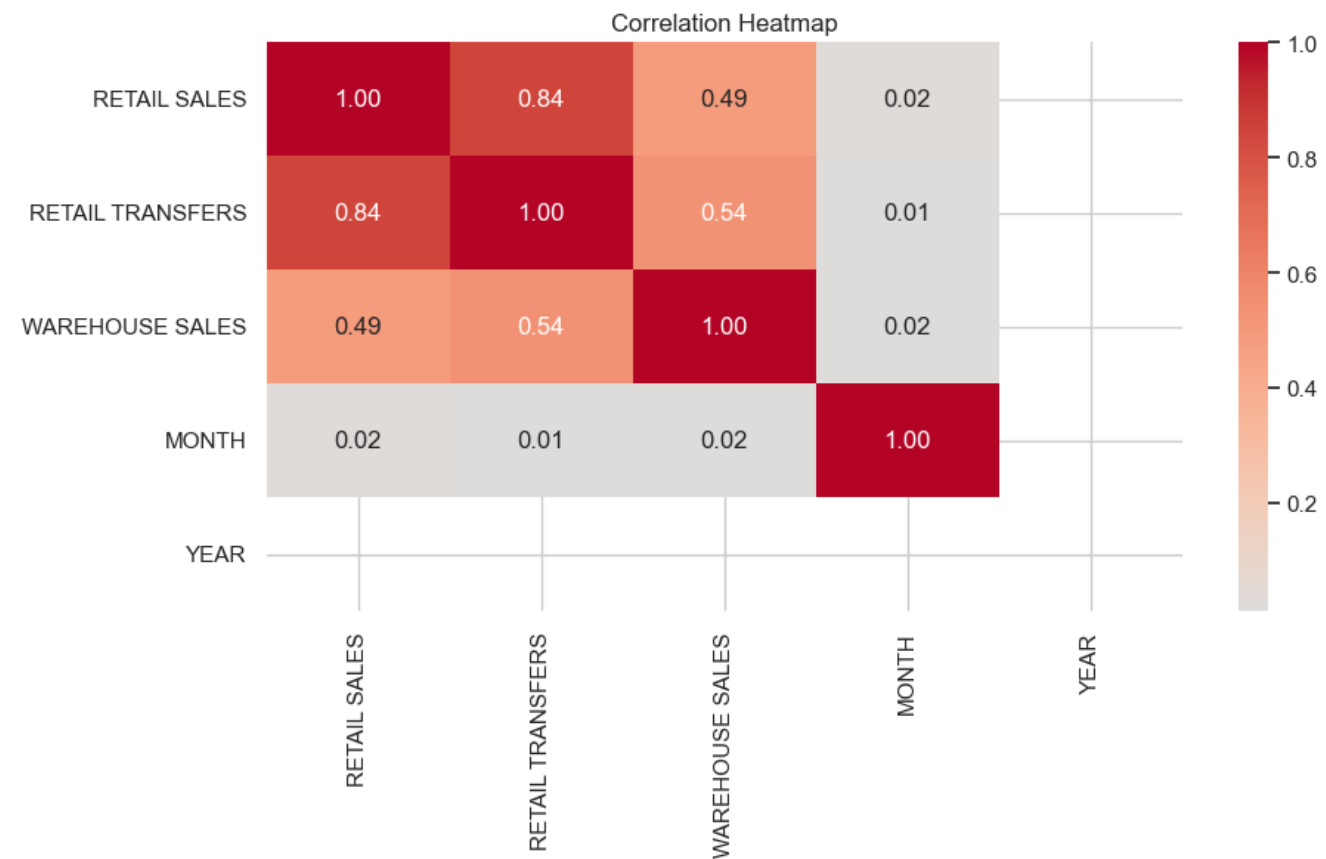
Histograms are used to inspect the distribution of retail sales. A log-transformed chart may help reveal patterns when sales are highly right-skewed. Distribution assumptions should be considered before interpreting statistical tests.



14. Correlation Analysis

Correlation measures the direction and strength of a linear relationship between numerical variables. A positive coefficient indicates that variables tend to increase together; a negative coefficient indicates an inverse relationship. Correlation alone does not establish causation.

	RETAIL SALES	RETAIL TRANSFERS	WAREHOUSE SALES	MONTH	YEAR
RETAIL SALES	1.000000	0.844597	0.491822	0.024374	NaN
RETAIL TRANSFERS	0.844597	1.000000	0.540071	0.011052	NaN
WAREHOUSE SALES	0.491822	0.540071	1.000000	0.017212	NaN
MONTH	0.024374	0.011052	0.017212	1.000000	NaN
YEAR	NaN	NaN	NaN	NaN	NaN



15. Regression Analysis

If WAREHOUSE SALES is available and has sufficient variation, linear regression is used to estimate RETAIL SALES from WAREHOUSE SALES. The model reports the intercept, slope coefficient, R-squared, mean absolute error (MAE), and root mean squared error (RMSE). The result is exploratory and should not be interpreted as proof of causation.

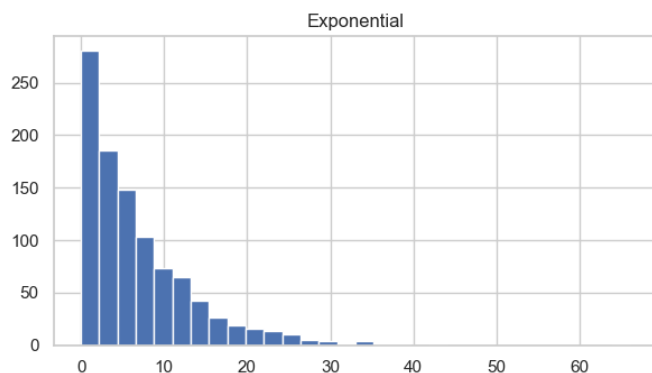
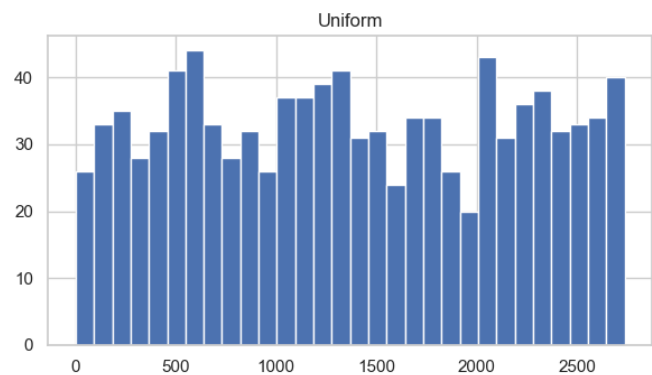
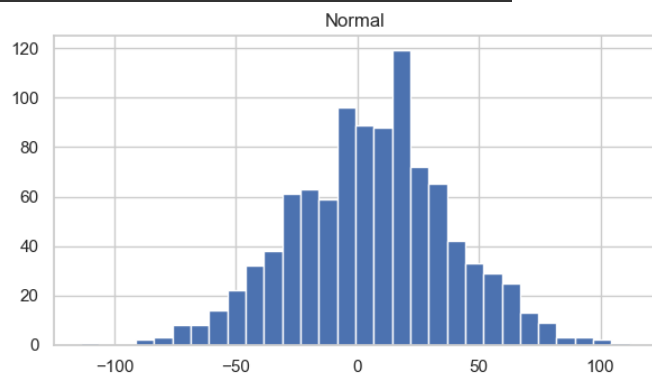
```
Regression equation: Retail Sales = 5.2999 + (0.0598 × Warehouse Sales)
R²: 0.24188926105786268
MAE: 9.033586641427723
RMSE: 28.80304621060166
```



16. Probability Distributions

The notebook calculates the empirical probability that retail sales exceed the observed median. It also demonstrates normal, uniform, and exponential random distributions. These generated distributions are teaching demonstrations and are not additional observed business records.

```
P(RETAIL SALES > median 0.160) = 0.4974
```



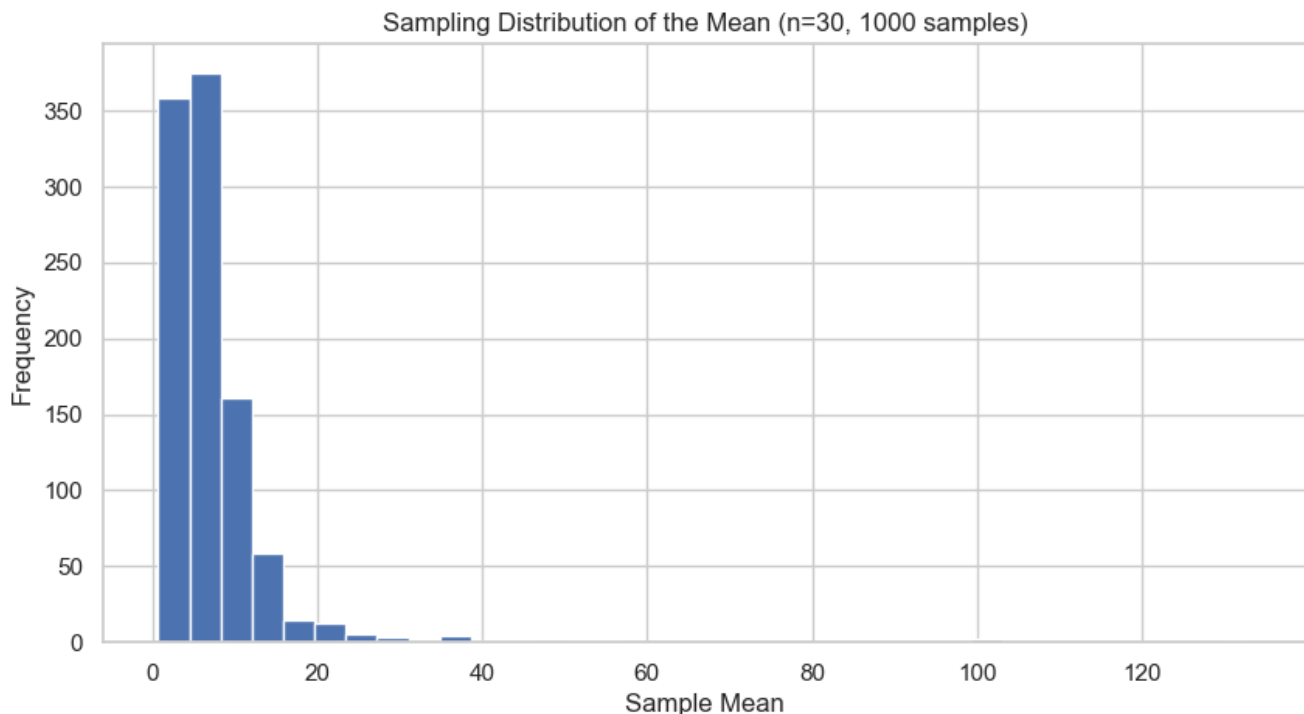
17. Decision-Making Under Uncertainty

Decision-making under uncertainty compares possible actions when demand or sales are variable. The inventory demonstration compares candidate stock levels using unit price, unit cost, holding cost, and shortage cost assumptions. These assumptions must be replaced with verified business values before using the results operationally.

	Scenario	Stock level	Mean units sold	Mean shortage	Mean leftover	Service level
0	Low stock (P50)	0.16	0.084525	6.855316	0.075475	0.502583
1	Medium stock (P75)	2.92	0.979039	5.960802	1.940961	0.750025
2	High stock (P90)	16.24	3.021642	3.918199	13.218358	0.900063

18. Sampling and Sampling Distribution

Repeated samples are drawn from the observed retail-sales values, and the mean of each sample is calculated. The distribution of these sample means illustrates sampling variability and the Central Limit Theorem.



19. Confidence Interval Estimation

Confidence intervals estimate a range of plausible values for the population mean based on the sample. The notebook reports 90%, 95%, and 99% confidence intervals using the t-distribution method.

Mean of sample means: 7.212029333333334			
Standard deviation of sample means: 7.879609106486997			
Approximate standard error: 6.0397464623195365			
	Confidence level	Lower	Upper
0	90%	6.625625	7.253967
1	95%	6.565434	7.314158
2	99%	6.447790	7.431802

20. Bootstrapping

Bootstrapping repeatedly samples observed sales values with replacement to estimate the uncertainty of the median. The notebook calculates a percentile-based 95% bootstrap confidence interval.

```
Observed median: 0.16
95% bootstrap confidence interval: (np.float64(0.16), np.float64(0.17))
```



21. Hypothesis Testing

Business Question

Is the mean retail sales value statistically different from a selected business benchmark?

Hypotheses

H0: The population mean retail sales equals the selected benchmark.

H1: The population mean retail sales differs from the selected benchmark.

Significance Level

$\alpha = 0.05$, unless another level is justified.

Statistical Test and Code

A one-sample t-test is included as a demonstration. Replace the placeholder benchmark in the notebook with a meaningful target before interpreting the result.

```
H0: Mean retail sales = 0.0
H1: Mean retail sales differs from 0.0
t-statistic: 36.334631279508635
p-value: 6.32161866278761e-283
Significance level: 0.05
Decision: Reject H0
Interpretation: Use a meaningful business benchmark before drawing a practical conclusion.
```

Decision and Business Interpretation

Reject H0 when $p\text{-value} < 0.05$; otherwise, do not reject H0. Explain the decision using the actual statistic and business benchmark. Statistical significance does not automatically imply business importance.

22. A/B Testing Design

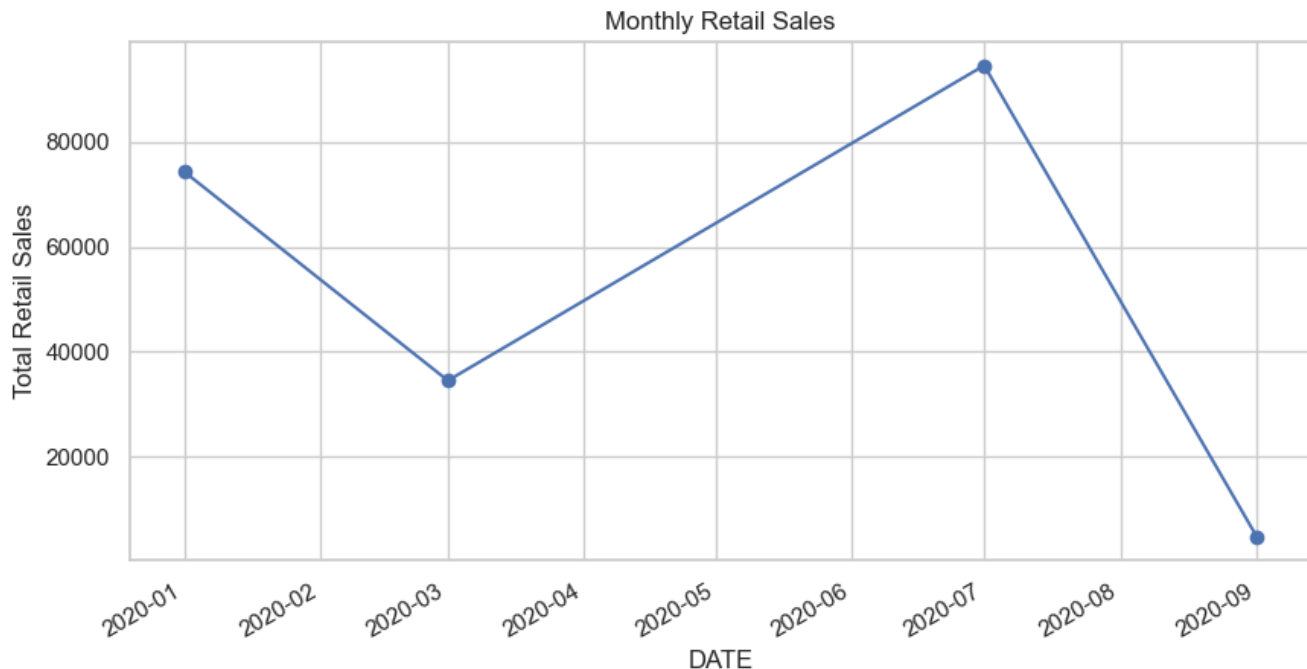
A/B testing could compare a current sales promotion (control) with a new promotion (treatment). The current dataset does not necessarily contain treatment assignments, so this section describes a proposed experiment rather than claiming that a real A/B test has been conducted.

- Business question: Does the new promotion increase sales compared with the current strategy?
- Control group: Eligible stores or products using the existing approach.
- Treatment group: Comparable stores or products using the new promotion.
- Response variable: Sales per store/product over a fixed period.
- Procedure: Define eligibility, randomize assignment where feasible, collect outcomes, and compare groups using a suitable statistical test.

23. Time-Series Analysis

If YEAR and MONTH are available, retail sales are aggregated by month and plotted over time. The data previously shown by the student contained YEAR = 2020; if the full dataset contains only one year or only one monthly period, forecasting may be limited or inappropriate.

Total Retail Sales	
DATE	
2020-01-01	74318.77
2020-03-01	34523.90
2020-07-01	94538.96
2020-09-01	4805.31



24. Forecast Accuracy

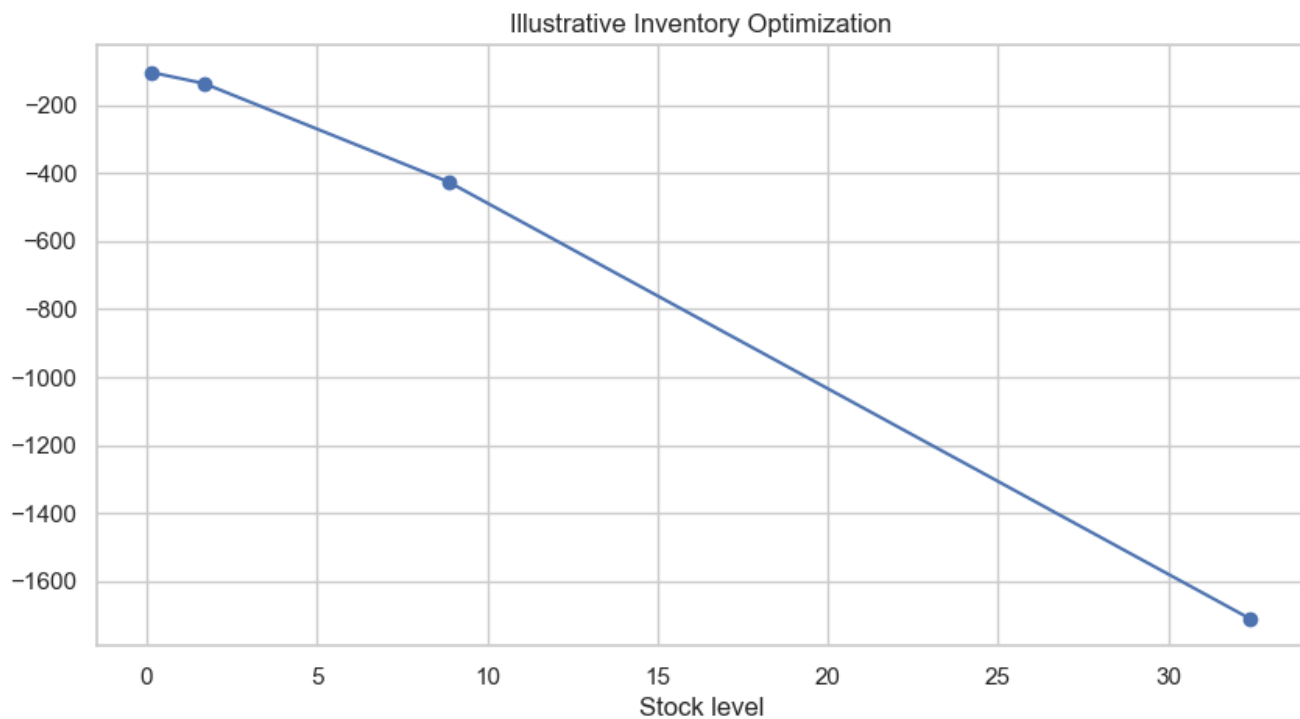
When enough monthly observations exist, the notebook compares exponential smoothing and ARIMA forecasts using a holdout sample. Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) are used to evaluate forecast accuracy. If insufficient time periods exist, record that forecasting was skipped instead of inventing results.

Forecast comparison skipped: only 4 monthly period(s). Use a dataset with more periods for meaningful forecasting.

25. Optimization Modeling

The inventory optimization demonstration compares candidate stock levels derived from observed sales quantiles. It estimates average simulated profit, leftover inventory, and shortage using stated assumptions. Results are illustrative until actual price and cost inputs are available.

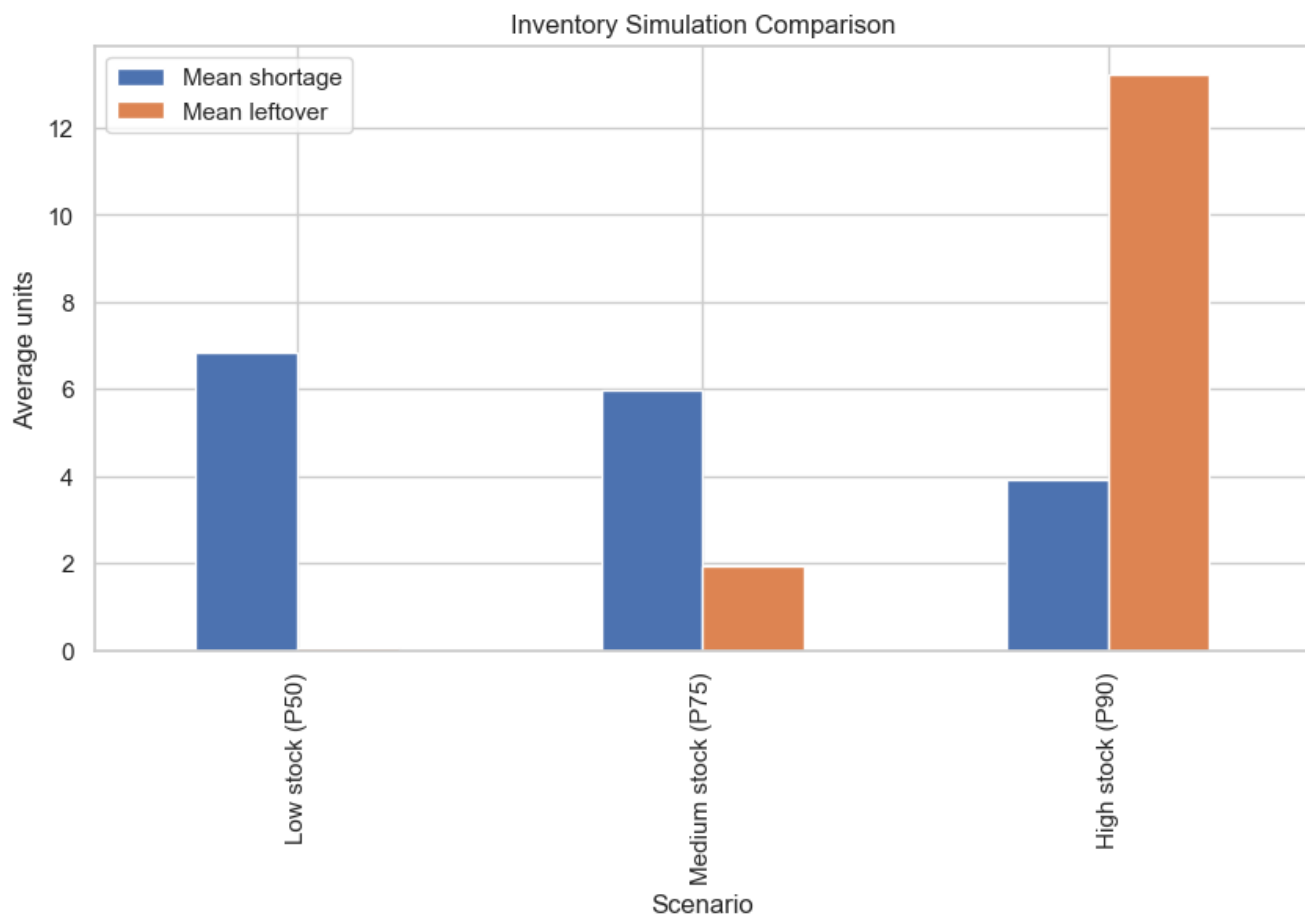
	Stock level	Average simulated profit	Average leftover	Average shortage
0	0.160	-104.354597	0.075475	6.855316
1	1.690	-136.676359	1.046073	6.295914
2	8.860	-425.796102	6.741654	4.821495
3	32.391	-1709.831679	28.226992	2.775833



26. Simulation Modeling

The simulation compares low-, medium-, and high-stock scenarios. Measures include average units sold, average shortage, average leftover inventory, and service level. Retail sales are used as a proxy for demand in this academic demonstration; this assumption should be validated before operational use.

	Scenario	Stock level	Mean units sold	Mean shortage	Mean leftover	Service level
0	Low stock (P50)	0.16	0.084525	6.855316	0.075475	0.502583
1	Medium stock (P75)	2.92	0.979039	5.960802	1.940961	0.750025
2	High stock (P90)	16.24	3.021642	3.918199	13.218358	0.900063



27. ANOVA

One-way ANOVA compares mean RETAIL SALES across ITEM TYPE groups.

H0: Mean retail sales are equal across all item types.

H1: At least one item type has a different mean retail sales value.

The notebook reports the F-statistic and p-value. At $\alpha = 0.05$, reject H0 if $p < 0.05$; otherwise, do not reject H0. Check group sizes, independence, and variance assumptions before interpreting the result.

```
H0: Mean retail sales are equal across item types.
H1: At least one item type has a different mean.
F-statistic: 123.52155338346167
p-value: 8.685629095379197e-180
Significance level: 0.05
Decision: Reject H0
```

28. Experimental Design

A proposed experiment could test whether a promotion changes sales. The factor is promotional strategy, with current and new promotion levels. The response variable is sales over a fixed period, and the experimental units may be stores or eligible product groups. Random assignment and consistent measurement windows should be used where feasible.

29. Factorial Experiment

A factorial analysis studies the main effects of two factors and their interaction. The proposed factors are ITEM TYPE and YEAR, with RETAIL SALES as the response. The screenshot shared earlier showed item types under YEAR = 2020. If the complete dataset contains only one year, the YEAR effect and ITEM TYPE $\times$ YEAR interaction cannot be estimated. Do not invent additional years; use genuine multi-year data.

Factorial ANOVA cannot test YEAR or ITEM TYPE × YEAR interaction.
Reason: the dataset contains fewer than two distinct years.
Do not invent year values. Use a multi-year dataset to complete this analysis.

30. Business Insights and Findings

Complete the findings after running the notebook. Each finding must refer to an actual statistic, table, chart, or model output.

1. Item type with the highest total sales.
2. Item type with the highest average sales.
3. Distribution skewness and presence of outliers.
4. Correlation between retail sales and warehouse sales/transfers.
5. Regression coefficient, R^2 , and prediction error.
6. Confidence interval for mean sales.
7. One-way ANOVA decision and p-value.
8. Forecast accuracy, only if enough monthly periods exist.
9. Inventory scenario with the best simulated profit under stated assumptions.
10. Limitations caused by the dataset's time coverage.

32. Business Recommendations

- Use item-type summaries to support category-level stock reviews.
- Investigate high sales variability before setting inventory buffers.
- Use correlation and regression results only with appropriate diagnostics and caveats.
- Use time-series forecasts only when sufficient reliable monthly observations are available.
- Replace illustrative inventory assumptions with actual selling, purchase, holding, and shortage costs.
- Monitor missing values, duplicate records, and inconsistent category labels.
- Use a dashboard to monitor sales performance and identify changes over time.
- Avoid causal claims unless supported by a suitable experimental design.

34. Conclusion

This project demonstrates how Business Analytics techniques can transform retail and warehouse sales records into descriptive summaries, visualizations, statistical tests, and decision-support scenarios. The final conclusion should summarize the strongest findings from the executed notebook and state the limitations of the dataset, particularly if the available data covers only one year.

35. Appendix

- A. Dataset information and sample records.
- B. Complete Python/Jupyter Notebook code.
- C. Data preprocessing outputs.
- D. EDA charts and descriptive-statistics tables.
- E. Correlation and regression outputs.
- F. Probability, sampling, confidence interval, and bootstrap outputs.
- G. Hypothesis test and ANOVA outputs.
- H. Forecasting and forecast-accuracy outputs, if supported.
- I. Optimization and simulation outputs.
- J. Business Intelligence dashboard screenshots.

Source Code:

```
# =====  
# BUSINESS ANALYTICS PROJECT  
# RETAIL AND WAREHOUSE SALES ANALYSIS  
# =====  
# =====  
# 1. IMPORT LIBRARIES AND LOAD DATASET  
# =====  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt
```

```

import seaborn as sns
import warnings

from scipy import stats

from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

import statsmodels.api as sm
from statsmodels.formula.api import ols

from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA

# Visualization settings
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 5)

```

```

# =====
# 2. LOAD DATASET
# =====

```

```
DATA_FILE = "Retail and warehouse Sale - Retail and warehouse Sale.csv"
```

```
df = pd.read_csv(DATA_FILE)
```

```
df.columns = df.columns.str.strip()
```

```

print("Dataset Name:", DATA_FILE)
print("Dataset Shape:", df.shape)

```

```

print("\nFirst Five Records:")
display(df.head())

```

```

# =====
# 3. DATASET INFORMATION
# =====

```

```
print("Number of Rows:", df.shape[0])
```

```
print("Number of Columns:", df.shape[1])
```

```

print("\nColumn Names:")
print(df.columns.tolist())

```

```

print("\nData Types:")
display(df.dtypes.to_frame("Data Type"))

print("\nMissing Values:")
display(df.isnull().sum().to_frame("Missing Values"))

print("\nDuplicate Records:")
print(df.duplicated().sum())

print("\nLast Five Records:")
display(df.tail())

```

```

# =====
# 4. DATA PREPROCESSING
# =====

```

```

required_columns = [
    "ITEM TYPE",
    "RETAIL SALES"
]

```

```

missing_columns = (
    set(required_columns)
    - set(df.columns)
)

```

```

if missing_columns:

```

```

    raise ValueError(
        f"Required columns missing: {missing_columns}"
    )

```

```

# Convert numerical columns
for col in [
    "YEAR",
    "MONTH",
    "RETAIL SALES",
    "RETAIL TRANSFERS",
    "WAREHOUSE SALES"
]:

```

```

    if col in df.columns:

```

```

        df[col] = pd.to_numeric(
            df[col],
            errors="coerce"
        )

```

```

# Clean ITEM TYPE
df["ITEM TYPE"] = (
    df["ITEM TYPE"]
    .astype("string")
    .str.strip()
)

# Remove missing values
df = df.dropna(
    subset=[
        "ITEM TYPE",
        "RETAIL SALES"
    ]
)

# Remove blank item types
df = df[
    df["ITEM TYPE"].str.len() > 0
]

```

```

print("Cleaned Dataset Shape:")
print(df.shape)

```

```

print("\nCleaned Dataset:")
display(df.head())

```

```

# =====
# 5. UNIQUE ITEM TYPES
# =====

```

```

print("Unique ITEM TYPE Categories:")

```

```

print(
    sorted(
        df["ITEM TYPE"]
        .dropna()
        .astype(str)
        .unique()
    )
)

```

```

# =====
# 6. ITEM TYPE SUMMARY
# =====

```

```

item_summary = df.groupby(
    "ITEM TYPE"
)["RETAIL SALES"].agg(

```

```

count="count",

mean="mean",

median="median",

total="sum",

std="std"

)

item_summary = item_summary.sort_values(
    "total",
    ascending=False
)

print("ITEM TYPE Summary:")

display(item_summary)

# =====
# 7. TOTAL RETAIL SALES BY ITEM TYPE
# =====

item_summary["total"].plot(
    kind="bar",
    title="Total Retail Sales by Item Type"
)

plt.xlabel("Item Type")

plt.ylabel("Total Retail Sales")

plt.xticks(
    rotation=40
)

plt.tight_layout()

plt.show()

# =====
# 8. AVERAGE RETAIL SALES BY ITEM TYPE
# =====

item_summary["mean"].sort_values(
    ascending=False
).plot(
    kind="bar",

```

```

    title="Average Retail Sales by Item Type"
)

plt.xlabel("Item Type")

plt.ylabel("Average Retail Sales")

plt.xticks(
    rotation=40
)

plt.tight_layout()

plt.show()

```

```

# =====
# 9. RETAIL SALES HISTOGRAM
# =====

```

```

df["RETAIL SALES"].hist(
    bins=40
)

plt.title(
    "Distribution of Retail Sales"
)

plt.xlabel(
    "Retail Sales"
)

plt.ylabel(
    "Frequency"
)

plt.tight_layout()

plt.show()

```

```

# =====
# 10. LOG-TRANSFORMED RETAIL SALES
# =====

```

```

log_sales = np.log1p(
    df["RETAIL SALES"].clip(
        lower=0
    )
)

plt.hist(
    log_sales,
    bins=40
)

```

```
)

plt.title(
    "Log-Transformed Retail Sales"
)

plt.xlabel(
    "log(1 + Retail Sales)"
)

plt.ylabel(
    "Frequency"
)

plt.tight_layout()

plt.show()
```

```
# =====
# 11. BOX PLOT OF RETAIL SALES BY ITEM TYPE
# =====
```

```
sns.boxplot(
    data=df,
    x="ITEM TYPE",
    y="RETAIL SALES"
)

plt.title(
    "Retail Sales by Item Type"
)

plt.xlabel(
    "Item Type"
)

plt.ylabel(
    "Retail Sales"
)

plt.xticks(
    rotation=40
)

plt.tight_layout()

plt.show()
```

```
# =====
# 12. SALES SHARE PIE CHART
# =====
```

```
item_summary["total"].plot(
```

```

    kind="pie",
    autopct="%1.1f%%"
)

plt.title(
    "Share of Total Retail Sales by Item Type"
)

plt.ylabel("")

plt.tight_layout()

plt.show()

```

```

# =====
# 13. DESCRIPTIVE STATISTICS
# =====

```

```

sales = df[
    "RETAIL SALES"
].dropna()

```

```

descriptive_statistics = pd.Series({

    "Count":
        sales.count(),

    "Mean":
        sales.mean(),

    "Median":
        sales.median(),

    "Mode":
        sales.mode().iloc[0]
        if not sales.mode().empty
        else np.nan,

    "Minimum":
        sales.min(),

    "Maximum":
        sales.max(),

    "Range":
        sales.max() - sales.min(),

    "Variance":
        sales.var(),

    "Standard Deviation":
        sales.std(),

```

```

        "Skewness":
            sales.skew()

    })

print("Descriptive Statistics:")

display(
    descriptive_statistics.to_frame(
        "Value"
    )
)

# =====
# 14. CORRELATION ANALYSIS
# =====

numeric_columns = [

    col

    for col in [

        "RETAIL SALES",
        "RETAIL TRANSFERS",
        "WAREHOUSE SALES",
        "MONTH",
        "YEAR"

    ]

    if col in df.columns

]

correlation_matrix = df[
    numeric_columns
].corr(
    numeric_only=True
)

print("Correlation Matrix:")

display(
    correlation_matrix
)

# =====
# 15. CORRELATION HEATMAP
# =====

```

```

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    center=0,
    fmt=".2f"
)

plt.title(
    "Correlation Heatmap"
)

plt.tight_layout()

plt.show()

```

```

# =====
# 16. RETAIL SALES VS WAREHOUSE SALES
# =====

```

if "WAREHOUSE SALES" in df.columns:

```

    scatter_data = df.dropna(
        subset=[
            "WAREHOUSE SALES",
            "RETAIL SALES"
        ]
    )

```

```

    plt.scatter(
        scatter_data["WAREHOUSE SALES"],
        scatter_data["RETAIL SALES"],
        alpha=0.35
    )

```

```

    plt.xlabel(
        "Warehouse Sales"
    )

```

```

    plt.ylabel(
        "Retail Sales"
    )

```

```

    plt.title(
        "Retail Sales vs Warehouse Sales"
    )

```

```

    plt.tight_layout()

    plt.show()

```

else:

```
print(
    "WAREHOUSE SALES column not available."
)
```

```
# =====
# 17. LINEAR REGRESSION
# =====
```

if "WAREHOUSE SALES" in df.columns:

```
regression_data = df.dropna(
    subset=[
        "WAREHOUSE SALES",
        "RETAIL SALES"
    ]
)
```

```
if (
    len(regression_data) >= 3
    and
    regression_data[
        "WAREHOUSE SALES"
    ].nunique() > 1
):
```

```
X = regression_data[
    ["WAREHOUSE SALES"]
]
```

```
y = regression_data[
    "RETAIL SALES"
]
```

```
regression_model = LinearRegression()
```

```
regression_model.fit(
    X,
    y
)
```

```
predictions = (
    regression_model
    .predict(X)
)
```

```
print(
    "Regression Equation:"
)
```

```
print(
    f"Retail Sales = "
    f"{regression_model.intercept_:.4f}"
    f" + "
    f"({regression_model.coef_[0]:.4f}"
    f" × Warehouse Sales)"
)
```

```
print(
    "\nR²:",
    r2_score(
        y,
        predictions
    )
)
```

```
print(
    "MAE:",
    mean_absolute_error(
        y,
        predictions
    )
)
```

```
print(
    "RMSE:",
    mean_squared_error(
        y,
        predictions
    ) ** 0.5
)
```

```
# Regression line
order = np.argsort(
    X[
        "WAREHOUSE SALES"
    ].to_numpy()
)
```

```
plt.scatter(
    X["WAREHOUSE SALES"],
    y,
    alpha=0.35
)
```

```
plt.plot(
    X[
        "WAREHOUSE SALES"
    ].to_numpy()[order],
    predictions[order]
)
```

```
plt.xlabel(
    "Warehouse Sales"
)
```

```
plt.ylabel(
    "Retail Sales"
)
```

```
plt.title(
    "Linear Regression"
)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
else:
```

```
print(
    "Regression skipped because "
    "there are insufficient observations "
    "or insufficient predictor variation."
)
```

```
else:
```

```
print(
    "Regression skipped because "
    "WAREHOUSE SALES is not available."
)
```

```
# =====
# 18. PROBABILITY ANALYSIS
# =====
```

```
median_sales = sales.median()
```

```
probability_above_median = (
    sales > median_sales
).mean()
```

```

print(
    "Median Retail Sales:",
    median_sales
)

print(
    "Probability of Retail Sales "
    "being greater than the median:"
)

print(
    probability_above_median
)

```

```

# =====
# 19. SIMULATED PROBABILITY DISTRIBUTIONS
# =====

```

```

rng = np.random.default_rng(42)

```

```

simulated_data = pd.DataFrame({

    "Normal":
        rng.normal(
            sales.mean(),
            sales.std()
            if sales.std() > 0
            else 1,
            1000
        ),

    "Uniform":
        rng.uniform(
            sales.min(),
            sales.max(),
            1000
        ),

    "Exponential":
        rng.exponential(
            max(
                sales.mean(),
                0.001
            ),
            1000
        )

})

```

```

simulated_data.hist(

```

```
    bins=30,  
    figsize=(12, 7)  
)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====  
# 20. SAMPLING DISTRIBUTION  
# =====
```

```
rng = np.random.default_rng(42)
```

```
sample_size = min(  
    30,  
    len(sales)  
)
```

```
if sample_size >= 2:
```

```
    sample_means = [  
        rng.choice(  
            sales.to_numpy(),  
            size=sample_size,  
            replace=True  
        ).mean()
```

```
        for _ in range(1000)
```

```
    ]
```

```
plt.hist(  
    sample_means,  
    bins=35  
)
```

```
plt.title(  
    "Sampling Distribution of Sample Means"  
)
```

```
plt.xlabel(  
    "Sample Mean"  
)
```

```
plt.ylabel(  
    "Frequency")
```

```
    "Frequency"  
)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
print(  
    "Mean of Sample Means:",  
    np.mean(sample_means)  
)
```

```
print(  
    "Standard Deviation of Sample Means:",  
    np.std(  
        sample_means,  
        ddof=1  
    )  
)
```

```
print(  
    "Approximate Standard Error:",  
    sales.std(  
        ddof=1  
    ) / np.sqrt(  
        sample_size  
    )  
)
```

```
else:
```

```
    print(  
        "Not enough observations "  
        "for sampling analysis."  
    )
```

```
# =====  
# 21. CONFIDENCE INTERVALS  
# =====
```

```
if len(sales) >= 2:
```

```
    confidence_intervals = []
```

```
    for confidence_level in [  
        0.90,  
        0.95,
```

```
0.99  
]:
```

```
lower, upper = stats.t.interval(  
    confidence_level,  
    df=len(sales) - 1,  
    loc=sales.mean(),  
    scale=stats.sem(  
        sales  
    )  
)
```

```
confidence_intervals.append({  
    "Confidence Level":  
        f"{confidence_level:.0%}",  
    "Lower Limit":  
        lower,  
    "Upper Limit":  
        upper  
})
```

```
confidence_table = pd.DataFrame(  
    confidence_intervals  
)
```

```
print(  
    "Confidence Interval Results:"  
)
```

```
display(  
    confidence_table  
)
```

```
else:
```

```
print(  
    "Not enough observations "  
    "for confidence interval."  
)
```

```

# =====
# 22. BOOTSTRAPPING
# =====

rng = np.random.default_rng(123)

bootstrap_medians = [

    np.median(

        rng.choice(
            sales.to_numpy(),
            size=len(sales),
            replace=True
        )

    )

    for _ in range(1000)

]

bootstrap_lower, bootstrap_upper = (
    np.percentile(
        bootstrap_medians,
        [2.5, 97.5]
    )
)

print(
    "Observed Median:",
    sales.median()
)

print(
    "95% Bootstrap Confidence Interval:"
)

print(
    bootstrap_lower,
    bootstrap_upper
)

plt.hist(
    bootstrap_medians,
    bins=30
)

```

```
plt.title(  
    "Bootstrap Distribution of Median Retail Sales"  
)
```

```
plt.xlabel(  
    "Bootstrap Median"  
)
```

```
plt.ylabel(  
    "Frequency"  
)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====  
# 23. HYPOTHESIS TESTING  
# =====
```

```
# Business benchmark  
# Replace this value with an appropriate  
# business benchmark before final submission.
```

```
benchmark = 0.0
```

```
t_statistic, p_value = stats.ttest_1samp(  
    sales,  
    popmean=benchmark,  
    nan_policy="omit"  
)
```

```
alpha = 0.05
```

```
print(  
    "Null Hypothesis (H0):"  
)
```

```
print(  
    f"Mean retail sales = {benchmark}"  
)
```

```
print(  

```

```

    "\nAlternative Hypothesis (H1):"
)

print(
    f"Mean retail sales differs from {benchmark}"
)

```

```

print(
    "\nT-Statistic:",
    t_statistic
)

```

```

print(
    "P-Value:",
    p_value
)

```

```

print(
    "Significance Level:",
    alpha
)

```

```

if p_value < alpha:

```

```

    print(
        "\nDecision: Reject H0"
    )

```

```

else:

```

```

    print(
        "\nDecision: Do Not Reject H0"
    )

```

```

# =====
# 24. TIME-SERIES ANALYSIS
# =====

```

```

if {
    "YEAR",
    "MONTH"
}.issubset(df.columns):

```

```

    time_series_data = df.dropna(

        subset=[
            "YEAR",
            "MONTH",
            "RETAIL SALES"

```

```

    ]

).copy()

# Create DATE column
time_series_data["DATE"] = pd.to_datetime(

    time_series_data[
        "YEAR"
    ].astype(int).astype(str)

    + "-"

    +

    time_series_data[
        "MONTH"
    ].astype(int).astype(str).str.zfill(2)

    + "-01",

    errors="coerce"
)

```

```

# Monthly sales
monthly_sales = (

    time_series_data

    .dropna(
        subset=["DATE"]
    )

    .groupby(
        "DATE"
    )["RETAIL SALES"]

    .sum()

    .sort_index()

)

```

```

print(
    "Monthly Retail Sales:"
)

```

```

display(
    monthly_sales.to_frame(
        "Total Retail Sales"
    )
)

```

```

    )
)

# Monthly sales graph
monthly_sales.plot(
    marker="o",
    title="Monthly Retail Sales"
)

plt.xlabel(
    "Month"
)

plt.ylabel(
    "Total Retail Sales"
)

plt.tight_layout()

plt.show()

else:

    print(
        "Time-series analysis requires "
        "YEAR and MONTH columns."
    )

# =====
# 25. FORECASTING AND FORECAST ACCURACY
# =====

if {
    "YEAR",
    "MONTH"
}.issubset(df.columns):

    if len(monthly_sales) >= 8:

        split = max(
            2,
            int(
                len(monthly_sales)
                * 0.80
            )
        )

```

```
train = monthly_sales.iloc[
    :split
]
```

```
test = monthly_sales.iloc[
    split:
]
```

```
forecast_results = []
```

```
# -----
# Exponential Smoothing
# -----
```

```
try:
```

```
    with warnings.catch_warnings():
```

```
        warnings.simplefilter(
            "ignore"
        )
```

```
        exponential_model = (
            ExponentialSmoothing(
                train,
                trend="add",
                seasonal=None
            ).fit()
        )
```

```
        exponential_forecast = (
            exponential_model
            .forecast(
                len(test)
            )
        )
```

```
        forecast_results.append({

            "Model":
                "Exponential Smoothing",

            "MAE":
                mean_absolute_error(
                    test,
                    exponential_forecast
                ),

            "RMSE":
```

```

        mean_squared_error(
            test,
            exponential_forecast
        ) ** 0.5
    })

```

except Exception as e:

```

    print(
        "Exponential Smoothing skipped:",
        e
    )

```

```

# -----
# ARIMA
# -----

```

try:

with warnings.catch_warnings():

```

    warnings.simplefilter(
        "ignore"
    )

```

```

    arima_model = ARIMA(
        train,
        order=(0, 1, 0)
    ).fit()

```

```

    arima_forecast = (
        arima_model
        .forecast(
            len(test)
        )
    )

```

forecast_results.append({

```

    "Model":
        "ARIMA(0,1,0)",

```

```

    "MAE":
        mean_absolute_error(
            test,
            arima_forecast
        ),

```

```

    "RMSE":

```

```

        mean_squared_error(
            test,
            arima_forecast
        ) ** 0.5
    })

```

except Exception as e:

```

    print(
        "ARIMA skipped:",
        e
    )

```

Display results
if forecast_results:

```

    forecast_table = pd.DataFrame(
        forecast_results
    ).sort_values(
        "RMSE"
    )

```

```

    print(
        "Forecast Accuracy Comparison:"
    )

```

```

    display(
        forecast_table
    )

```

else:

```

    print(
        "Forecasting skipped."
    )

```

```

    print(
        "Reason: insufficient monthly "
        "observations."
    )

```

else:

```

    print(
        "Forecasting requires YEAR "
        "and MONTH columns."
    )

```

```
# =====  
# 26. ONE-WAY ANOVA  
# =====
```

```
groups = [  
    group[  
        "RETAIL SALES"  
    ].dropna().to_numpy()  
  
    for _, group  
    in df.groupby(  
        "ITEM TYPE"  
    )  
  
    if len(  
        group[  
            "RETAIL SALES"  
        ].dropna()  
    ) >= 2  
]  
  
if len(groups) >= 2:  
  
    try:  
  
        f_statistic, p_value = (  
            stats.f_oneway(  
                *groups  
            )  
        )  
  
        alpha = 0.05  
  
        print(  
            "One-Way ANOVA"  
        )  
  
        print(  
            "\nNull Hypothesis (H0):"  
        )  
  
        print(  
            "Mean retail sales are equal "  
            "across all item types."
```

```
)
```

```
print(  
    "\nAlternative Hypothesis (H1):"  
)
```

```
print(  
    "At least one item type has "  
    "a different mean."  
)
```

```
print(  
    "\nF-Statistic:",  
    f_statistic  
)
```

```
print(  
    "P-Value:",  
    p_value  
)
```

```
print(  
    "Significance Level:",  
    alpha  
)
```

```
if p_value < alpha:
```

```
    print(  
        "\nDecision: Reject H0"  
    )
```

```
    print(  
        "Conclusion: There is a "  
        "statistically significant "  
        "difference among item-type "  
        "mean retail sales."  
    )
```

```
else:
```

```
    print(  
        "\nDecision: Do Not Reject H0"  
    )
```

```
print(  
    "
```

```

        "Conclusion: There is not "
        "enough statistical evidence "
        "to conclude that the item-type "
        "means differ."
    )

```

```

except ValueError as e:

```

```

    print(
        "ANOVA Error:",
        e
    )

```

```

else:

```

```

    print(
        "ANOVA skipped because at least "
        "two item types with sufficient "
        "observations are required."
    )

```

```

# =====
# 27. FACTORIAL ANOVA
# ITEM TYPE × YEAR
# =====

```

```

if (
    "YEAR" in df.columns
    and
    df["YEAR"].nunique(
        dropna=True
    ) >= 2
):

```

```

    factorial_data = df.dropna(

```

```

        subset=[
            "YEAR",
            "ITEM TYPE",
            "RETAIL SALES"
        ]

```

```

    ).copy()

```

```

    factorial_data["YEAR"] = (
        factorial_data[
            "YEAR"
        ].astype(str)
    )

```

```
factorial_data["ITEM TYPE"] = (
    factorial_data[
        "ITEM TYPE"
    ].astype(str)
)
```

```
try:
```

```
    factorial_model = ols(
        'Q("RETAIL SALES") ~ '
        'C(Q("ITEM TYPE")) * C(YEAR)',
        data=factorial_data
    ).fit()
```

```
    factorial_anova = (
        sm.stats.anova_lm(
            factorial_model,
            typ=2
        )
    )
```

```
    print(
        "Factorial ANOVA Results:"
    )
```

```
    display(
        factorial_anova
    )
```

```
except Exception as e:
```

```
    print(
        "Factorial ANOVA could not "
        "be calculated."
    )
```

```
    print(
        "Error:",
        e
    )
```

```
else:
```

```
print(
    "Factorial ANOVA cannot be performed."
)
```

```
print(
    "Reason: The dataset contains "
    "fewer than two distinct years."
)
```

```
print(
    "The YEAR main effect and "
    "ITEM TYPE × YEAR interaction "
    "cannot be estimated."
)
```

```
# =====
# 28. INVENTORY OPTIMIZATION
# =====
```

```
demand = sales.clip(
    lower=0
).to_numpy()
```

```
if len(demand) > 0:
```

```
    # Candidate inventory levels
    candidate_stock = np.unique(
```

```
        np.maximum(
```

```
            0,
```

```
            np.quantile(
```

```
                demand,
```

```
                [
```

```
                    0.50,
```

```
                    0.70,
```

```
                    0.85,
```

```
                    0.95
```

```
                ]
```

```
            )
```

```
        )
```

```
    )
```

```

# Illustrative assumptions
unit_price = 100.0

unit_cost = 60.0

holding_cost = 5.0

shortage_cost = 15.0

optimization_results = []

for stock_level in candidate_stock:

    units_sold = np.minimum(

        demand,

        stock_level

    )

    leftover_inventory = np.maximum(

        stock_level - demand,

        0

    )

    shortage = np.maximum(

        demand - stock_level,

        0

    )

    profit = (

        units_sold

        * unit_price

        - stock_level

        * unit_cost

        - leftover_inventory

        * holding_cost

```

```

        - shortage
        * shortage_cost
    )

    optimization_results.append({

        "Stock Level":
            stock_level,

        "Average Simulated Profit":
            profit.mean(),

        "Average Leftover":
            leftover_inventory.mean(),

        "Average Shortage":
            shortage.mean()

    })

    optimization_table = (
        pd.DataFrame(
            optimization_results
        )
        .sort_values(
            "Average Simulated Profit",
            ascending=False
        )
    )

    print(
        "Inventory Optimization Results:"
    )

    display(
        optimization_table
    )

    # Optimization graph
    optimization_table.plot(

        x="Stock Level",

        y="Average Simulated Profit",

        marker="o",

        legend=False,

```

```

        title="Inventory Optimization"
    )

    plt.xlabel(
        "Stock Level"
    )

    plt.ylabel(
        "Average Simulated Profit"
    )

    plt.tight_layout()

    plt.show()

else:

    print(
        "Inventory optimization skipped."
    )

# =====
# 29. INVENTORY SIMULATION
# =====

demand = sales.clip(
    lower=0
).to_numpy()

if len(demand) > 0:

    simulation_results = []

    scenarios = [

        ("Low Stock (P50)", 0.50),

        ("Medium Stock (P75)", 0.75),

        ("High Stock (P90)", 0.90)

    ]

    for scenario_name, percentile in scenarios:

```

```

stock_level = float(
    np.quantile(
        demand,
        percentile
    )
)

units_sold = np.minimum(
    demand,
    stock_level
)

shortage = np.maximum(
    demand - stock_level,
    0
)

leftover = np.maximum(
    stock_level - demand,
    0
)

service_level = (
    demand <= stock_level
).mean()

simulation_results.append({
    "Scenario":
        scenario_name,

    "Stock Level":
        stock_level,

    "Mean Units Sold":
        units_sold.mean(),

    "Mean Shortage":
        shortage.mean(),

    "Mean Leftover":

```

```

        leftover.mean(),

        "Service Level":
            service_level

    })

```

```

simulation_table = pd.DataFrame(
    simulation_results
)

```

```

print(
    "Inventory Simulation Results:"
)

```

```

display(
    simulation_table
)

```

```

# Simulation chart
simulation_table.set_index(
    "Scenario"
)[

    [
        "Mean Shortage",
        "Mean Leftover"
    ]

].plot(
    kind="bar",
    title="Inventory Simulation Comparison"
)

```

```

plt.xlabel(
    "Scenario"
)

```

```

plt.ylabel(
    "Average Units"
)

```

```

plt.tight_layout()

```

```

plt.show()

```

else:

```
print(
    "Inventory simulation skipped."
)

# =====
# 30. EXPORT CLEANED DATASET
# =====

df.to_csv(
    "cleaned_retail_warehouse_data.csv",
    index=False
)

print(
    "Cleaned dataset exported successfully."
)

print(
    "File: cleaned_retail_warehouse_data.csv"
)

# =====
# END OF BUSINESS ANALYTICS PROJECT
# =====
```